



Oil Terminal Management System (OTMS)

Technical Documentation Manual

Created by Praveen Christuraj

Generated on 2025-11-26

2. System Architecture Overview

OTMS follows a modular architecture. Each feature page is isolated into its own Python file, with shared logic encapsulated in core helpers. The main entry point `main_app.py` controls routing, authentication, and session management.

All operational pages (Tank, Tanker, YADE, FSO, Vessel, etc.) are stored in the `app_pages/` folder. Each exposes a `render_*` function that the main app invokes based on sidebar selection.

Configuration modules (`location_config`, `dashboard_config`, `fso_config`) hold per-location and per-dashboard settings, allowing location-aware control of visibility and behavior.

```
if page == 'Tank Transactions': render_tank_transactions_page(sess, user,
location_id)
```

3. Core Components

- `db.py` – creates SQLAlchemy sessions and manages local SQLite ↔ SQL Server bridge.
- `security.py` – handles authentication, session expiry, audit logging, and password policy.
- `permission_manager.py` – defines role-based access for operators, supervisors, managers, and admins.
- `auth.py` – login / logout endpoints and credential verification.
- `task_manager.py` – scheduler for background jobs like auto-backup and reconciliation.
- `recycle_bin.py` – safe archival of deleted objects with restore support.
- `logger.py` – central error logger with timestamped file output.

```
from db import get_session with get_session() as s: SecurityManager.log_audit(s,
user, 'UPDATE', resource_type='Tank', resource_id='T001')
```

4. Helper Modules

Helper utilities keep calculations and presentation reusable:

- `utils_calc.py` – contains full API ↔ Density ↔ VCF ↔ LT conversion formulas.
- `unique_id_generator.py` – produces IDs like AGGU-20250109-0001 to guarantee daily uniqueness.
- `timezone_utils.py` – converts UTC ↔ Africa/Lagos time for audit consistency.
- `ui.py` – shared Streamlit header layout.
- `get_browser_info.py`, `ip_service.py`, `health_check.py` – diagnostic utilities.

```
def calculate_vcf(api60, temp_f): temp_diff = temp_f - 60.0 rho60 = (141.5 * 999.012)
/ (api60 + 131.5) alpha = 341.0957 / (rho60 ** 2) return
round(math.exp(-alpha*temp_diff - 0.8*alpha*alpha*temp_diff*temp_diff), 5)
```

5. Data Models & ORM Layer

`models.py` defines over 40 ORM classes such as Tank, TankTransaction, TankerTransaction, YadeVoyage, FSOOperation, and AuditLog. Relationships are declared with SQLAlchemy to ensure referential integrity.

Each table includes `created\_by`, `updated\_by`, and timestamp fields for audit purposes.

```
class Tank(Base): __tablename__ = 'tanks' id = Column(Integer, primary_key=True) name
= Column(String) capacity_bbl = Column(Float)
```

6. Operational Pages

Every operational page corresponds to a workflow:

- Tank Transactions – records dip readings and computes TOV/GOV/GSV/NSV/LT/MT.
- Tanker Transactions – manages truck loading bays, destinations, and seal grids.
- YADE Transactions – handles barge voyages and auto-generates TOA PDFs.
- FSO Operations – controls floating-storage receipts, exports, and variances.
- Vessel Operations – logs shuttle vessel loading and discharging activities.

```
if st.session_state.get('auth_user')['role'] == 'admin-it': st.error('■ Access
Denied: Admin-IT users do not have operational access.')
```

7. Calculations & Conversions

Core formulas implemented in `utils\_calc.py`:

- Density @ 60 °F = SG × Water60; SG = 141.5 / (API + 131.5)
- API from Density = 141.5 / SG – 131.5
- VCF = $\exp(-\alpha\Delta T - 0.8\alpha^2\Delta T^2)$, where $\alpha = 341.0957 / \rho_{60}^2$
- LT = NSV × Table 11 Factor; MT = LT × 1.01605
- NSV = GSV – (GSV × BS&W; %)

```
def compute_all_for_tank_tx(...): TOV = tank_volume_from_dip_cm(sess, tank_name,
dip_cm) FW = free_water_from_water_cm(sess, tank_name, water_cm) GOV = TOV - FW api60
= api_observed_to_api60(api_obs, sample_temp_f) vcf = calculate_vcf(api60,
tank_temp_f) GSV = round(GOV * vcf, 0)
```

8. Dashboard & Reporting Engine

OTMS dashboards are dynamic: layout, cards, and colors are stored in `dashboard\_config.py` as JSON.

`dashboard\_utils.py` and `dashboard\_widgets.py` render KPIs, trends, and tank visuals.

`report\_engine.py` and `report\_customization.py` allow managers to design new reports directly from the UI.

```
report = ReportEngine(session).generate('daily_tank_summary', start, end)
```

9. Security & Audit Framework

Security stack:

- Role-based access from `permission\_manager.py`.
- Two-Factor Auth (TOTP) via `twofa.py` – compatible with Google Authenticator.

- `SecurityManager.log\_audit()` records all CRUD events.
- Session timeout enforcement for inactivity.

All actions are traceable by user, timestamp, and location.

```
if TwoFactorAuth.verify_token(session, user.id, token): login_success() else:
    st.error('Invalid 2FA code')
```

10. Automation & Backup System

`backup\_manager.py` and `backup\_scheduler.py` automate daily database dumps with retention policies. `task\_manager.py` runs scheduled housekeeping and notification tasks.

```
BackupManager.create_backup('D:/Project OTMS-Rebuild/backups')
```

11. Utility & Support Tools

Diagnostics and support scripts include:

- `health\_check.py` – verifies DB connectivity and disk space.
- `ip\_service.py` – resolves client IPs for audit context.
- `get\_browser\_info.py` – fetches browser User-Agent for session logs.

```
ua = get_browser_user_agent() logger.info(f'Browser: {ua}')
```

12. Configuration & Setup Scripts

Initial setup utilities:

- `setup\_fso\_permissions.py` – pre-grants FSO page access to specific sites (Agge, Utapate, Lagos).
- `location\_manager.py` and `location\_config.py` – CRUD for location data and settings.
- `material\_balance\_config.py` and `material\_balance\_calculator.py` – define reconciliation logic.

```
if __name__ == '__main__': print('■ Setting up FSO permissions...')
    setup_fso_permissions()
```

13. Appendices and Future Roadmap

Appendix A – Database Schema Overview (ER diagram available in Power BI dashboards).

Appendix B – Role Matrix: Operator < Supervisor < Manager < Admin-Ops < Admin-IT.

Appendix C – Future Enhancements: REST API gateway, mobile views, cloud sync scheduler, and live sensor feeds.

Thank you for using Project OTMS — a complete terminal management solution crafted by Praveen Christuraj.

End of Technical Manual